Estrategias de programación y estructuras de datos

Grado en Ingeniería Informática

Grado en Tecnologías de la Información

2

Estrategias de programación

Algoritmos iterativos y recursivos

UNED

ETS de Ingeniería Informática

Estrategias de programación

Objetivos generales

Objetivos

- Entender las principales características del diseño recursivo
- Entender las ventajas y desventajas del diseño recursivo
- Aprender a diseñar algoritmos recursivos
- Advertir su importancia en el contexto de las abstracciones de datos
- Entender las principales características del diseño iterativo
- Entender las ventajas y desventajas del diseño iterativo
- Aprender a diseñar algoritmos iterativos
- Advertir su importancia en el contexto de las abstracciones de datos
- Aprender a transformar algoritmos recursivos en iterativos

Tema	Descripción	Capítulos Libro	Páginas
2	Programación recursiva	7.1-7.4	165-188, 205-211

Estrategias de programación

Introducción

Todo algoritmo pretende la realización de algún tipo de tarea o cálculo. Las más de las veces, si dicha tarea no es trivial, esto implica repetir un conjunto de pasos sencillos un cierto número de veces hasta que se cumpla cierta condición, que se conoce como condición de terminación. Esencialmente, hay dos formas de diseñar algoritmos que desarrollen tareas repetitivas, que veremos en este tema ya que representan estrategias diferentes para resolver el mismo problema: la recursividad y la iteración

Estrategias de programación

I. Diseño recursivo de algoritmos

El problema se resuelve por el diseño de funciones que recurren sobre si mismas de forma directa o indirecta. Su construcción se basa en la elección de una familia de casos base, triviales de resolver y una colección de casos recursivos que converjan hacia los casos base y que en suma den cobertura a todo el espectro de posibles parámetros de entrada

II. Diseño iterativo de algoritmos

El problema se resuelve por aplicación de sentencias de control de flujo de programa sobre estructuras de datos que representan la entrada o salida del mismo. En este caso la llamada a otra funciones dentro de una función se debe a criterios de descomposición modular y no se aplica recursividad para resolver el problema

Estrategias de programación

Introducción

Diseño recursivo

Diseño iterativo

I. Diseño recursivo

Los algoritmos de diseño recursivo permiten resolver un problema invocando dentro del cuerpo de una función – directa o indirectamente – nuevamente a la propia función sobre un conjunto de parámetros actuales diferentes. Un diseño adecuado de este tipo de algoritmos debe garantizar que la ejecución de la función con unos parámetros adecuados converja a la solución de manera que se garantice la finalización del mismo

```
int factorial (int n) {
   if (n == 0) return 1;
   else return n * factorial (n – 1);
}
```

Para resolver la función factorial se vuelve a invocar a la función factorial con valor n – 1 y se devuelve el resultado de multiplicar el valor devuelto de esta invocación por n

```
Factorial (5) = 5 * factorial (4)
              = 5 * 4 * factorial (3)
              = 5 * 4 * 3 factorial (2)
              = 5 * 4 * 3 * 2 * factorial (1)
              = 5 * 4 * 3 * 2 * 1 * factorial (0)
              = 5 * 4 * 3 * 2 * 1 * 1
              = 120
```

Estrategias de programación

Introducción

Diseño recursivo

Diseño iterativo

I. Diseño iterativo

Los algoritmos iterativos son aquellos que se construyen a partir de una secuencia ordenada de instrucciones de alto nivel. Un subconjunto de estas instrucciones permiten alterar el flujo de ejecución. En concreto se distinguen instrucciones de control de flujo condicional – que seleccionan un camino de ejecución de entre varios posibles – e iterativo – que repiten la ejecución de un bloque de instrucción mientras se satisfagan ciertas condiciones ambientales. Un diseño adecuado de este tipo de algoritmos debe garantizar la terminación del mismo

```
int factorial (int n) {
   int index = 1;
   int result = 1;
   while (index <= n) {
      result = result * index;
      index ++;
   }
}
```

Para resolver la función factorial se procede iterativamente acumulando en la variable result el producto del contador index de cada iteración por el resultado acumulado anterior

Factorial (5)					
index	1	2	3	4	5
result	1	2	6	24	120

Estrategias de programación

¿Qué son los algoritmos recursivos?

Un algoritmo recursivo es aquel que se invoca a si mismo dentro de su propio cuerpo con unos parámetros actuales distintos. Estructuralmente cada función está formada por una colección de alternativas disjuntas llamadas casos con guardas expresadas en términos de los parámetros del problema. Los casos base resuelven el problema de forma inmediata mientras que los otros recurren compositivamente sobre la función

Esquema general de un algoritmo recursivo

Función recursiva

```
Tr función (Tx x) {
  if (esCasoBase1 (x)) return <<resolución inmediata 1>>
    ...
  if (esCasoBaseN (x)) return <<resolución inmediata N>>

  if (esCasoRecursivo1 (x)) return <<resolución recursiva convergente 1>>
    ...
  if (esCasoRecursivoM (x)) return <<resolución recursiva convergente M>>
}
```

Casos base

Casos recursivos

Casos

El diseño requiere la definición de una colección de casos que den cobertura a toda entrada legítima de parámetros

Guarda

La guarda del caso refleja las condiciones ambientales que deben satisfacerse para ejecutar el caso. Están expresadas en términos de los parámetros y deben ser disjuntas entre sí

Resolución

La resolución de un caso indica las acciones que deben efectuarse para resolver el caso. Si es base la resolución es inmediata si es recursiva consiste en la aplicación de una composición de llamadas recursivas

Estrategias de programación

Diseño de algoritmos recursivos

Como hemos visto, la tarea de diseño de una función recursiva consiste en la definición de la colección de casos base y casos recursivos que resuelvan satisfactoriamente el problema. A continuación discutimos el correcto diseño de cada tipo de caso

Diseño de casos base

Un caso base representa el final de una tarea, ya que no requiere de nuevas llamadas recursivas para su resolución debido a su sencillez. Cuando diseñamos los casos base de un algoritmo recursivo debemos asegurarnos de que las guardas de todos ellos son disjuntas entre sí y de que en suma dan cobertura a todas las posibles invocaciones provenientes de algún caso recursivo

Caso base

Toda invocación recursiva termina por aplicación del único caso base con guarda n==0. A veces es posible balancear los casos de un diseño recursivo entre los base y los propiamente recursivos sin impacto en la solución. Considere que otra solución hubiera sido definir el caso base con la guarda n<2

```
int factorial (int n) {
   if (n == 0) return 1;
   else return n * factorial (n – 1);
}
```

Estrategias de programación

Diseño de algoritmos recursivos

Como hemos visto, la tarea de diseño de una función recursiva consiste en la definición de la colección de casos base y casos recursivos que resuelvan satisfactoriamente el problema. A continuación discutimos el correcto diseño de cada tipo de caso

Diseño de casos recursivos

Un caso recursivo consta de tres elementos: una división del problema en uno o más subproblemas, una invocaciones recursiva para resolver cada uno de estos subproblemas y una combinación de los resultados de los subproblemas para obtener, a la vuelta de las invocaciones recursivas, el resultado del problema original. Este esquema termina si la división del problema converge hacia alguno de los casos base de la función

Caso recursivo

Aquí el único caso recursivo es aquel con guarda implícita n>0. Su resolución consiste en una división convergente hacia n==0 debido a la expresión de división del problema

```
int factorial (int n) {
   if (n == 0) return 1;
   else return n * factorial (n – 1);
}
```

División

La función de división genera a cada paso de recursión un subproblema de tamaño una unidad menor que converge a 0

Recursión

La recursión se aplica para resolver el subproblema de tamaño n-1

Combinación

La función de combinación en este caso que permite resolver el problema a partir de los resultados parciales de cada subproblema es la operación producto

Estrategias de programación

Diseño de algoritmos recursivos

Como hemos visto, la tarea de diseño de una función recursiva consiste en la definición de la colección de casos base y casos recursivos que resuelvan satisfactoriamente el problema. A continuación discutimos el correcto diseño de cada tipo de caso

Diseño de casos recursivos

Un caso recursivo consta de tres elementos: una división del problema en uno o más subproblemas, una invocaciones recursiva para resolver cada uno de ellos y una combinación de los resultados de los subproblemas para obtener, a la vuelta de las invocaciones recursivas, el resultado del problema original.

Clasificaciones de recursividad

I. Por la forma de recursión

El número de invocaciones realizadas en cada caso recursivo marca un criterio de clasificación

Recursividad simple	*return n * f (n-1)*
Recursividad múltiple	*return f (n-1) + … + f (n-k)*

II. Por la forma de partición

La forma de dividir el problema en subproblemas es otro criterio de clasificación

Partición por substracción	*return n * f (n-1)*
Partición por división	*return f (n/2)*

III. Por la forma de combinación

La forma de combinar los resultados parciales de la resolución de cada subproblema es el tercer criterio

Recursividad final	*return f (n/2)*
Recursividad no final	*return n * f (n-1)*

Estrategias de programación

Diseño de algoritmos recursivos

Como hemos visto, la tarea de diseño de una función recursiva consiste en la definición de la colección de casos base y casos recursivos que resuelvan satisfactoriamente el problema. A continuación discutimos el correcto diseño de cada tipo de caso

Diseño de recursividad por inmersión

A veces el diseño de una función recursiva se expresa como una invocación concreta de una función más general que incluye parámetros adicionales. Esta técnica, conocida por el nombre de diseño por inmersión, se aplica por distintas cuestiones que discutiremos a continuación

> La inmersión es una estrategia de diseño mediante la cual una función se expresa como una invocación particular de otra función más general con parámetros adiciones

Función sumergida

La función sumergida se expresa como una invocación concreta de la función inmersora más general

```
int sumaTodos (int v[]) {
   return sumaDesde (v, 0);
}

int sumaDesde (int v[], int index) { ... }
```

Función inmersora

La función inmersora sumerge a la original puesto que, al ser más general, con una invocación particular de ésta se da cobertura al problema

Estrategias de programación

Diseño de algoritmos recursivos

Como hemos visto, la tarea de diseño de una función recursiva consiste en la definición de la colección de casos base y casos recursivos que resuelvan satisfactoriamente el problema. A continuación discutimos el correcto diseño de cada tipo de caso

Diseño de recursividad por inmersión

A veces el diseño de una función recursiva se expresa como una invocación concreta de una función más general que incluye parámetros adicionales. Esta técnica, conocida por el nombre de diseño por inmersión, se aplica por distintas cuestiones que discutiremos a continuación

Táctica de inmersión

I. Inmersión de parámetros

La inmersión de parámetros se refiere a la aplicación de la técnica de inmersión sobre los parámetros de entrada de la misma. Existen 2 tipos dentro de esta categoría

I.I. Parámetros de recorrido

Se incluyen parámetros en la función de inmersora que controlan el recorrido de los elementos de una estructura de datos

I.II. Parámetros acumuladores

Se incluyen parámetros en la función de inmersora que acumulan resultados inter-medios

II. Inmersión de resultados

La inmersión de resultados consiste en la aplicación de la técnica de la inmersión para acumular en parámetros el resultado del problema en llamada recursiva

Estrategias de programación

Diseño de algoritmos recursivos

Como hemos visto, la tarea de diseño de una función recursiva consiste en la definición de la colección de casos base y casos recursivos que resuelvan satisfactoriamente el problema. A continuación discutimos el correcto diseño de cada tipo de caso

Diseño de recursividad por inmersión

A veces el diseño de una función recursiva se expresa como una invocación concreta de una función más general que incluye parámetros adicionales. Esta técnica, conocida por el nombre de diseño por inmersión, se aplica por distintas cuestiones que discutiremos a continuación

Objetivos de inmersión

I. Inmersión por diseño

Se aplican técnicas de inmersión con el objeto de definir una recursión que de otro modo no sería posible articular

II. Inmersión por eficiencia

Se aplican técnicas de inmersión con el fin de conseguir una ejecución más eficiente de la función en la resolución del problema

III. Inmersión por recursión final

Se aplican técnicas de inmersión con el ánimo de transformar la recursividad en recursión final

Estrategias de programación

Diseño de algoritmos recursivos

Como hemos visto, la tarea de diseño de una función recursiva consiste en la definición de la colección de casos base y casos recursivos que resuelvan satisfactoriamente el problema. A continuación discutimos el correcto diseño de cada tipo de caso

Diseño de recursividad por inmersión

A veces el diseño de una función recursiva se expresa como una invocación concreta de una función más general que incluye parámetros adicionales. Esta técnica, conocida por el nombre de diseño por inmersión, se aplica por distintas cuestiones que discutiremos a continuación

		Táctica		
		Recorrido	**Acumulación**	**Resultados**
Objetivos	**Diseño**	Se aplica para obtener recurrencias con estructuras de datos estáticas o parámetros constantes durante la ejecución	-	-
	Eficiencia	-	Esta táctica de inmersión se aplica para precalcular y acumular un valor de entrada de manera que se ahorre en tiempo o memoria	Esta táctica de inmersión se aplica para calcular y utilizar un resultado parcial, que viene de una llamada posterior en la cadena recursiva
	R. Final	-	Se incluyen parámetros de acumulación para evitar tener que combinar tras la ejecución de las recursiones de cada subproblema	-

Estrategias de programación

Diseño de algoritmos recursivos

Como hemos visto, la tarea de diseño de una función recursiva consiste en la definición de la colección de casos base y casos recursivos que resuelvan satisfactoriamente el problema. A continuación discutimos el correcto diseño de cada tipo de caso

Ejercicios

I. Función factorial

Diseñe una función que calcule el factorial de un número. Determine el tipo de recursividad de que se trata según las 3 clasificaciones anteriores

II. Serie de fibonacci

Cada valor de la serie de fibonacci se obtiene a partir de la suma de los dos anteriores. Los dos primeros elementos de la serie son 1

III. Potencia de un número

Calcule la n-esima potencia de un número mediante el diseño de una función recursiva. Clasifique el diseño con respecto a los criterios anteriores

IV. Suma de N naturales

Diseñe una función recursiva que calcule la suma de los n primeros números naturales. ¿Es recursiva simple? ¿es final?

V. Suma de un vector

Diseñe una función recursiva que calcule la suma de los n primeros elementos de un vector de enteros. Clasifique la función

VI. Producto escalar

Diseñe una función recursiva que calcule el producto escalar de dos vectores de enteros. Clasifique la función

VII. Búsqueda en vector

Diseñe una función recursiva que busque un elemento dentro de un vector de enteros. Clasifique la función

VIII. Elementos repetidos

Diseñe un predicado recursivo que determine si un vector de enteros contiene o no algún elemento repetido

IX. Máximo común divisor

Diseñe una función recursiva que calcule el máximo común divisior entre dos enteros por aplicación del algoritmo de Euclides

Estrategias de programación

¿Qué son los algoritmos iterativos?

En los algoritmos iterativos los problemas se resuelven mediante la escritura de una colección de instrucciones de propósito específico que se ejecutan de manera secuencial según el orden en que se han escrito. Algunas de estas instrucciones permiten alterar el control secuencial de la ejecución para articular flujos iterativos o condicionales de ejecución

Esquema general de un algoritmo iterativo

```
int está (T[] v, T e) {
   int index = 0;
   boolean found = false;
   while (!found && index < v.length) {
      found = (v [index] == e);
      if (!found) index++;
   }
   if (found) return index;
   else return -1;
}
```

Inicialización

La inicialización es el paso preliminar para proceder con el diseño iterativo de algoritmos

Iteración

Los algoritmos iterativos basan su funcionamiento en iteraciones de bloques de código generalmente aplicadas sobre elementos de estructuras de datos

Bifurcación

Las instrucciones de control de flujo condicional permiten ejecutar sentencias solo bajo determinadas condiciones ambientales

Secuenciamiento

La característica esencial del diseño iterativo radica en el concepto de secuencia de instrucciones ejecutadas en orden y en el uso de variables para acumular estados parciales

Estrategias de programación

Diseño de algoritmos iterativos

El diseño iterativo utiliza instrucciones que prescriben cómo realizar los cálculos o tareas necesarias para llegar al resultado buscado. Estas instrucciones se ejecutan de acuerdo a flujos de ejecución, que pueden ser secuenciales, bifurcativos e iterativos. La utilización de variables y la sentencia de asignación juegan un papel preponderante

Elementos de un algoritmo iteratitivo

Elementos de los algoritmos iterativos

I. Sentencias de asignación

La sentencia de asignación juegan un papel preponderante en el diseño iterativo ya que permite acumular cálculos parciales que representan el estado del algoritmo en ese punto

II. Sentencias de control de flujo

La ejecución de instrucciones puede articularse en secuencia, bifurcativamente o en iteración. Existen sentencias que permiten determinar el control de flujo

A. Encadenamiento secuencial

Las instrucciones se encadenan secuencialmente para que se ejecuten una detrás de otra

B. Control de flujo condicional

Las instrucciones se agrupan en dos bloques que se ejecutan alternativamente si se satisface una condición

C. Control de flujo iterativo

Las instrucciones se agrupan en un bloque que se ejecuta iterativamente mientras se satisface una condición

Estrategias de programación

Diseño de algoritmos iterativos

El diseño iterativo utiliza instrucciones que prescriben cómo realizar los cálculos o tareas necesarias para llegar al resultado buscado. Estas instrucciones se ejecutan de acuerdo a flujos de ejecución, que pueden ser secuenciales, bifurcativos e iterativos. La utilización de variables y la sentencia de asignación juegan un papel preponderante

Sentencia de asignación

En el diseño iterativo las variables representan almacenes de datos en memoria que contienen el valor resultante del cómputo de una expresión. Este valor representa parcialmente el estado del algoritmo en tiempo de ejecución en función de los elementos de entrada

```
Sentencia;
variable = expresión;
Sentencia;
```

Estado

Cada sentencia supone un cambio potencial en el estado de la máquina. La asignación permite recogerlo parcialmente en el valor de la variable

Asignación

A diferencia de cómo ocurre en algoritmos recursivos las sentencias de asignación permiten apoyarse en variables para que sean referidas posteriormente desde otros puntos del algoritmo

Expresión y variable

La variable es una referencia simbólica a un especio reservado en memoria. La expresión una fórmula que permite calcular un valor en función de otras variables y constantes

Estrategias de programación

Diseño de algoritmos iterativos

El diseño iterativo utiliza instrucciones que prescriben cómo realizar los cálculos o tareas necesarias para llegar al resultado buscado. Estas instrucciones se ejecutan de acuerdo a flujos de ejecución, que pueden ser secuenciales, bifurcativos e iterativos. La utilización de variables y la sentencia de asignación juegan un papel preponderante

Encadenamiento secuencial

El modelo de ejecución por defecto es la ejecución secuencial. Según éste, las instrucciones son lanzadas a ejecución de acuerdo al orden en que aparecen correlativamente escritas en el algoritmo

```
Sentencia;
Sentencia;
Sentencia;
```

Encadenamiento secuencial

El delimitador de sentencias separa una sentencia de la siguiente dentro de un bloque de instrucciones secuencial y puede ser considerado como el *operador* que articula el encadenamiento secuencial

Ejecución en secuencia

La ejecución en secuencia se alinea con la idea de que cada sentencia realiza un paso del algoritmo hacia la obtención del resultado final y con la descomposición funcional descendente del paradigma estructurado

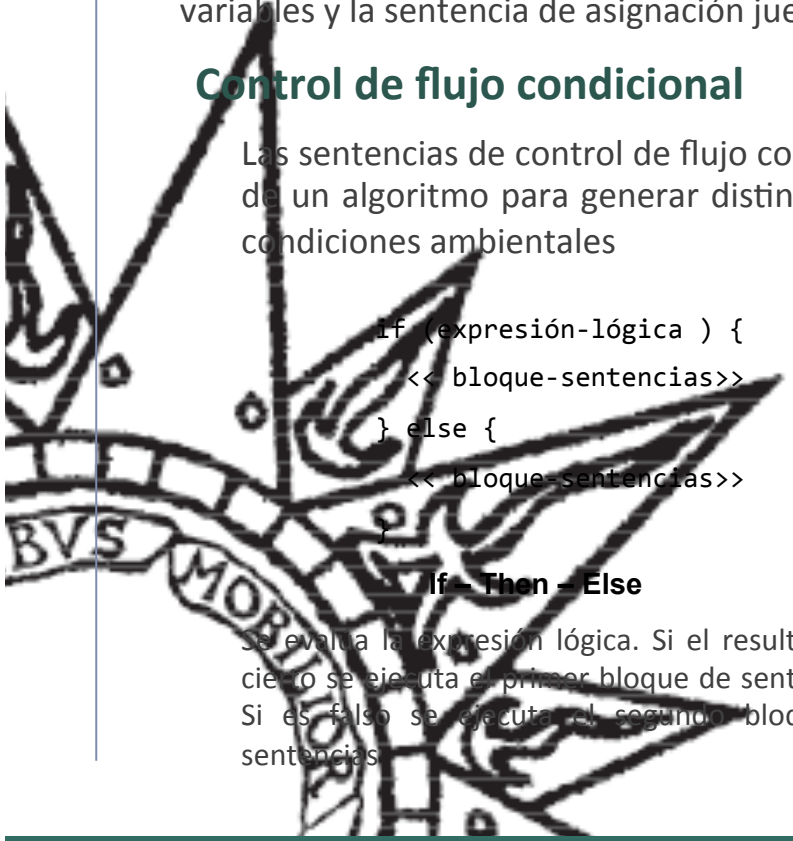
Estrategias de programación

Diseño de algoritmos iterativos

El diseño iterativo utiliza instrucciones que prescriben cómo realizar los cálculos o tareas necesarias para llegar al resultado buscado. Estas instrucciones se ejecutan de acuerdo a flujos de ejecución, que pueden ser secuenciales, bifurcativos e iterativos. La utilización de variables y la sentencia de asignación juegan un papel preponderante

Control de flujo condicional

Las sentencias de control de flujo condicional alteran la ejecución normal – secuencial – de un algoritmo para generar distintas bifurcaciones ejecutables en función de ciertas condiciones ambientales



```
if (expresión-lógica ) {
   << bloque-sentencias>>
} else {
   << bloque-sentencias>>
}
```

If – Then – Else

Se evalúa la expresión lógica. Si el resultado es cierto se ejecuta el primer bloque de sentencias. Si es falso se ejecuta el segundo bloque de sentencias

```
switch (expresión) {
  case cte-1: <<bloque-sentencias>> break;
  case cte-2: <<bloque-sentencias>> break;
  ...
  default: <<bloque-sentencias>>
}
```

Switch - Case

Se evalúa la expresión. Si coincide con la constante del caso i se ejecuta el bloque de sentencias adjunto. Si no se ajusta a ningún caso se ejecuta el bloque de sentencias default

Estrategias de programación

Diseño de algoritmos iterativos

El diseño iterativo utiliza instrucciones que prescriben cómo realizar los cálculos o tareas necesarias para llegar al resultado buscado. Estas instrucciones se ejecutan de acuerdo a flujos de ejecución, que pueden ser secuenciales, bifurcativos e iterativos. La utilización de variables y la sentencia de asignación juegan un papel preponderante

Control de flujo iterativo

Las sentencias de control de flujo iterativo alteran la ejecución normal – secuencial – de un algoritmo para generar distintas iteraciones sobre un cuerpo de sentencias. El número de iteraciones depende de condiciones ambientales

```
for (asignación-índice;
      expresión-límite;
      expresión-incremento) {
  << bloque-sentencias>>
}
```

For

Iteración de bloque de sentencias controlada por índice. El valor inicial se indica en asignación-índice, el incremento en expresión-incremento y termina cuando expresión-límite se hace falso

```
<<inicialización>>
while (expresión-lógica ) {
  << bloque-sentencias>>
  << incremento>>
}
```

While

Iteración de bloque se sentencias mientras expresión-lógica sea cierta. Inicialización externa al bucle. Incremento explícito típicamente al final del bloque de sentencias

```
<<inicialización>>
do {
  << bloque-sentencias>>
  << incremento>>
} while (expresión-lógica );
```

Do - While

Iteración de bloque de sentencias. Inicialización exterior. Incremento típicamente al final del bloque. Comprobación lógica al final de cada iteración. Una ejecución al menos

Estrategias de programación

Diseño de algoritmos iterativos

El diseño iterativo utiliza instrucciones que prescriben cómo realizar los cálculos o tareas necesarias para llegar al resultado buscado. Estas instrucciones se ejecutan de acuerdo a flujos de ejecución, que pueden ser secuenciales, bifurcativos e iterativos. La utilización de variables y la sentencia de asignación juegan un papel preponderante

Ejercicios

Encuentre una solución iterativa para cada uno de los problemas que se presentaron con anterioridad

I. **Función factorial**
II. **Serie de fibonacci**
III. **Potencia de un número**
IV. **Suma de N naturales**
V. **Suma de un vector**
VI. **Producto escalar**
VII. **Búsqueda en vector**
VIII. **Elementos repetidos**
IX. **Máximo común divisor**

```
// Pre: -1 < j <= length
public int sumaV (int[] v) {
  int j = 0;
  int s = 0;
  while (j < v.length) {
    s = s + v[j];
    j++;
}
  return s;
}
                    length
// Post: s = Σ v[i]
                    i=0
```

Traducción de algoritmos recursivos a iterativos

Los algoritmos recursivos aunque elegantes y eficaces han presentado problemas históricos de computabilidad ya que el soporte por parte de los compiladores a la invocación recursiva de programas es relativamente reciente. Merece la pena discutir procedimientos de transformación de algoritmos recursivos a iterativos

Traducción de algoritmos con recursividad final

```
public int sumaV (int[] v, int j, int w) {
 if (j >= n) return w
 else return sumaV (v, j+1, v[j] + w)
}
```

```
public int sumaVit (int [n] v) {
  int j = 0;
  int w = 0;

  while (j < n) {
    w = w + v[j];
    j = j + 1;
  }
  return w;
}
```

Los parámetros de entrada se substituyen por variables locales con el valor que recibirían en la llamada inicial (j). El parámetro acumulador (w) juega el papel del resultado parcial inicialmente 0.

Dentro del bucle se calcula el valor, en cada paso, de las variables de resultado parcial w y de recorrido j

Estrategias de programación

Traducción de algoritmos recursivos a iterativos

Los algoritmos recursivos aunque elegantes y eficaces han presentado problemas históricos de computabilidad ya que el soporte por parte de los compiladores a la invocación recursiva de programas es relativamente reciente. Merece la pena discutir procedimientos de transformación de algoritmos recursivos a iterativos

Traducción de algoritmos con recursividad no final

```
public int factorial( int n ) {
   if (n == 0) return 1;
   else return  n * factorial ( n - 1 );
 }
```

```
public int factIt (int n) {
  int i = n;
  while (i > 0) {
    i = i - 1
  }
  int r = 1;
  while (i < n) {
    r = r * i;
    i++;
  }
  return r
}
```

Buscar en i el final de la cadena de llamadas descendente. Se aplica en cada vuelta el sucesor de los parámetros (n-1) hasta encontrar el equivalente al tamaño del caso trivial (0).

Se usa r para acumular el resultado. Se inicializa al resultado del caso base. Se usa la variable i como objeto de comparación del segundo bucle. Se computa el resultado parcial aplicando la función de combinación (*) por el valor del parámetro. Al final se devuelve el resultado

Traducción de algoritmos recursivos a iterativos

Los algoritmos recursivos aunque elegantes y eficaces han presentado problemas históricos de computabilidad ya que el soporte por parte de los compiladores a la invocación recursiva de programas es relativamente reciente. Merece la pena discutir procedimientos de transformación de algoritmos recursivos a iterativos

Traducción de algoritmos mediante el uso de una pila

```
int invertir (int n, int r) {
  if (n == 0) return r;
  return invertir ((n / 10), r * 10 + n % 10);
  }
```

```
int invertirIt (int n) {
  int j = n;
  Stack <int> p=new Stack <int>();
  while (j > 10) {
    p.push (j);
    n = n / 10;
  }
  int k = j;
  while (!p.isEmpty ()) {
    j = p.top();
    p.pop();
    k = k + (j % 10);
  }
  return k  }
```

Inicialización de j como variable local que representa el valor inicial del parámetro y de p como pila vacía de trabajo

Apilamos el valor en curso de j para recuperarlo a la vuelta y calculamos el sucesos (n/10)

Hemos terminado la cadena descendente de llamadas. Ahora preparamos la cadena ascendente de vuelta. k llevará el resultado, que parte del valor de j cuando termina el bucle de ida

Estrategias de programación

Catálogo de algoritmos

Aunque encontrar un catálogo de algoritmos canónicos y recurrentes es complejo dado su gran diversidad, si relacionamos el diseño de algoritmos con los principales tipos abstractos de datos que se presentaron en el tema anterior, es posible encontrar 3 categorías generales en la que éstos caen principalmente de acuerdo a su propósito

Catálogo de algoritmos

I. Algoritmos de recorrido

El propósito de los algoritmos de recorrido es encontrar una expresión lineal de los elementos almacenados en la estructura de datos. Dependiendo de ésta puede haber varias estrategias de recorrido alternativas. El resultado es una colección ordenada de los elementos de la estructura

II. Algoritmos de búsqueda

El propósito de los algoritmos de búsqueda es encontrar una elemento de entre todos los elementos almacenados en la estructura de datos. Dependiendo de ésta existen distintas estrategias con complejidad viceversa. El resultado es un valor de verdad indicado si se encuentra el elemento o un índice de posición

III. Algoritmos de ordenación

El propósito de los algoritmos de ordenación es reorganizar los elementos de la estructura de datos para disponerlos de acuerdo a cierto criterio de orden. Para poder articular una ordenación sobre una estructura de datos los elementos deben ser comparables y la estructura prestarse al criterio de orden

Estrategias de programación

Catálogo de algoritmos

La naturaleza iterativa o recursiva de los algoritmos quede frecuentemente inducida por la forma en que el perfil de las operaciones da acceso a los datos. A lo largo de este curso repasaremos los principales tipos de algoritmos – recorrido, búsqueda y ordenación – aplicables sobre cada uno de los tipos abstractos de datos que estudiaremos

	Recorrido	Búsqueda	Ordenación
Listas*	Recorrido directo Recorrido inverso	Búsqueda secuencial	Inserción Merge sort
Pilas	Recorrido desde la cima	Búsqueda secuencial	-
Colas	Recorrido hacia el final	Búsqueda secuencial	-
Arboles	Preorden, inorden, postorden Recorrido en anchura	Búsqueda binaria	Ordenación

* Se refiere a listas, listas doblemente enlazadas y listas circulares

EJEMPLOS

Concepto de recursión

- Cuando un subprograma (función o procedimiento) hace uso de sí mismo se dice que es un *subprograma recursivo*

- **Ejemplo:** Factorial de un número

$$n! = n \times (n-1) \times (n-2) \times \ldots \times 4 \times 3 \times 2 \times 1 =$$
$$n \times ((n-1) \times (n-2) \times \ldots \times 4 \times 3 \times 2 \times 1) =$$
$$n \times (n-1)!$$

Tipos de recursividad

- **Recursividad lineal (k = 1):** Caso particular → Una sola llamada recursiva
  - Problemas de reducción (simplificación) del problema a 1. Por ejemplo, factorial de un número o la búsqueda binaria
- **Recursividad no lineal (k > 1)** → Más de una llamada recursiva
  - Por ejemplo, la sucesión de Fibonacci, donde k = 2 o la multiplicación de enteros muy grandes, donde k = 3

Recursividad de subprogramas: Terminación

- La ejecución de un algoritmo recursivo debe terminar en algún momento
- Para ello hace falta que las llamadas recursivas internas del subprograma a sí mismo deben estar controladas con una estructura condicional
- Se distinguen dos casos:
  - **Base:** La solución se obtiene directamente sin necesidad de llamadas recursivas
    - **Ejemplo:** 0! = 1 y 1! = 1, casos n = 0 o n = 1, respectivamente
  - **Recursión:** Se necesitan invocaciones recursivas para obtener la solución
    - **Ejemplo:** n! = n x (n-1)!, caso n > 1

Fundamentos matemáticos: Inducción matemática

- Las demostraciones por *inducción matemática* se usan habitualmente para demostrar teoremas que cumplen los números enteros
- Una demostración por **inducción matemática**:
  - Muestra primero que el teorema es cierto para algunos casos simples, y después
  - Muestra como extender la certeza indefinidamente

Inducción matemática: Pasos

- **Base:** Caso sencillo que se puede demostrar directamente, por definición
- **Paso inductivo:** La hipótesis de inducción supone que el teorema es cierto para un caso arbitrario k, demostrándose que a partir de esta hipótesis, se puede probar que el teorema es cierto para el siguiente caso (k + 1)
- **Conclusión:** Entonces la afirmación es cierta para todos los casos

Inducción matemática: Ejemplo

- La suma S(n) de los n primeros números naturales: 1 + 2 + 3 + … + n es:

$$S(n) = n * (n + 1) / 2$$

- **Caso base:** S(1) = 1
- **Hipótesis de inducción:** Suponemos cierto para un valor k > 1. Es decir, S(k) = k * (k + 1) / 2
- Entonces para k + 1, calculamos:

S(k + 1)

= 1 + 2 + 3 + … + k + (k + 1)	[Por definición]
= (1 + 2 + 3 + … + k) + (k + 1)	[Agrupando términos]
= S(k) + (k + 1)	[Substituyendo]
= (k * (k + 1) / 2) + (k + 1)	[Hipótesis de inducción]
= (k + 1) * (k / 2 + 1)	[Sacando factor común]
= (k + 1) * (k + 2) / 2, c.q.d.	[Operando]

Ejemplo 0: Función Suma

- Función recursiva para calcular la suma S(n) de los n primeros números naturales

$$S(n) = 1 + 2 + 3 + \ldots + n-1 + n$$

- **Definición recursiva:**
  - $S(1) = 1$
  - $S(n) = S(n-1) + n,\ n > 1$

n	S(n)
1	1
2	1 + 2 = 3
3	1 + 2 + 3 = 3 + 3 = 6
4	1 + 2 + 3 + 4 = 6 + 4 = 10
5	1 + 2 + 3 + 4 + 5 = 10 + 5 = 15
6	1 + 2 + 3 + 4 + 5 + 6 = 15 + 6 = 21
7	1 + 2 + 3 + 4 + 5 + 6 + 7 = 21 + 7 = 28
…	…

Ejemplo 0.1: Función Suma recursiva

```
public static long s(int n) {
// Calcula la suma S(n) = 1 + 2 + 3 + ... + n, n ≥ 1
   if (n == 1)
        return 1;
   else
        return s(n-1) + n;
}
```

n	S(n)
1	1
2	1 + 2 = 3
3	1 + 2 + 3 = 3 + 3 = 6
4	1 + 2 + 3 + 4 = 6 + 4 = 10
5	1 + 2 + 3 + 4 + 5 = 10 + 5 = 15
6	1 + 2 + 3 + 4 + 5 + 6 = 15 + 6 = 21
7	1 + 2 + 3 + 4 + 5 + 6 + 7 = 21 + 7 = 28
...	...

¿Cómo funciona?

- Java implementa los métodos mediante una pila de activación
- La terminación de los métodos se produce en sentido inverso a la invocación de los mismos
- Cuando se llama un método se coloca un nuevo registro en la cima de la pila, lo cual hace que sea el método actualmente activo
- Cuando termina se desapila la cima de la pila y el registro de activación que queda en la cima contiene los valores previos
- **Inconvenientes:**
  - Se produce una **sobrecarga en espacio** debido a la memoria necesaria para almacenar un registro de activación para cada método invocado → Coste espacial
  - La **sobrecarga en tiempo** producida al ejecutar una llamada a un método corresponde a la ejecución de las operaciones de apilar y desapilar sobre la pila interna del sistema → Coste temporal
- **Solución:**
  - Los programas recursivos se pueden implementar de forma iterativa con una pila explícita
  - Se espera que use algo menos de espacio y más rápido, pero el código será más largo y complejo → En contra de la claridad y sencillez

Estrategias de programación

Demasiada recursión puede ser peligrosa

- En ocasiones, los algoritmos que se expresan de forma natural (por definición) en modo recursivo deben reescribirse sin recursión para conseguir mayor eficiencia
- No se debe usar la recursión en sustitución de un simple bucle
- **Ejemplo:** La sucesión de Fibonacci

Ejemplo 0.2: Función Suma iterativa

```
public static long s(int n) {
// Calcula la suma S(n) = 1 + 2 + 3 + … + n, n ≥ 1
   long suma = 1;

   for (i = 2; i <= n; i++)
        suma += i;
   return suma;
}
```

n	s(n)
1	1
2	1 + 2 = 3
3	1 + 2 + 3 = 6
4	1 + 2 + 3 + 4 = 10
5	1 + 2 + 3 + 4 + 5 = 15
6	1 + 2 + 3 + 4 + 5 + 6 = 21
7	1 + 2 + 3 + 4 + 5 + 6 + 7 = 28
…	…

Ejemplo 1: Función de Fibonacci

- La **sucesión de Fibonacci** se define como:
  - fib(0) = fib(1) = 1
  - fib(n) = fib(n-1) + fib(n-2), si n ≥ 2
- Los primeros términos son:
  - 1, 1, 2, 3, 5, 8 , 13, 21, 34, 55, 89, 144, 233, 377, …

Estrategias de programación

Ejemplo 1.1: Función de Fibonacci recursiva

```
public static long fib(int n) {
// Obtiene el n-ésimo número de
// Fibonacci, n ≥ 1

        if (n == 0 || n == 1)
                return 1;
        else
                return fib(n-1) + fib(n-2);
}
```

n	fib(n)
0	1
1	1
2	1 + 1 = 2
3	2 + 1 = 3
4	3 + 2 = 5
5	5 + 3 = 8
6	8 + 5 = 13
7	13 + 8 = 21
8	21 + 13 = 34
9	34 + 21 = 55
...	...

Ejemplo 1.2: Función de Fibonacci iterativa

```
public static long fib(int n) {
// Obtiene el n-ésimo número de Fibonacci, n ≥ 1
    long x,y,z;
    int i;

    if (n == 0 || n == 1)
        return 1;
    else {
        x = 1;
        y = 1;
        for (i = 2; i <= n; i++) {
            z = x + y;
            x = y;
            y = z;
        }
        return z;
    }
}
```

n	fib(n)
0	1
1	1
2	1 + 1 = 2
3	2 + 1 = 3
4	3 + 2 = 5
5	5 + 3 = 8
6	8 + 5 = 13
7	13 + 8 = 21
8	21 + 13 = 34
9	34 + 21 = 55
...	...

Ejemplo 2: Función Factorial

- La **función factorial de un natural, n!** = n x (n-1) x (n-2) x … x 4 x 3 x 2 x 1, se define de forma recursiva como n! = n x (n-1)!:

  – factorial(0) = 1

  – factorial(1) = 1

  – factorial(n) =

  n * factorial(n-1), si n ≥ 2

n	n!
0	1
1	1
2	2 * 1 = 2
3	3 * 2 * 1 = 3 * 2 = 6
4	4 * 3 * 2 * 1 = 4 * 6 = 24
5	5 * 4 * 3 * 2 * 1 = 5 * 24 = 120
6	6 * 5 * 4 * 3 * 2 * 1 = 6 * 120 = 720
7	7 * 6 * 5 * 4 * 3 * 2 * 1 = 7 * 720 = 5040
…	…

Ejemplo 2.1: Función Factorial recursiva

```
public static long factorialRecursivo(int n) {
// Calcula el factorial de un natural n, n!

   if (n <= 1)
        return 1;
   else
        return n * factorialRecursivo(n-1);
}
```

Ejemplo 2.2: Función Factorial iterativa

```
public static long factorialIterativo(int n) {
// Calcula el factorial de un natural n, n!

    long factorial = 1;

    for (i = 2; i <= n; i++)
        factorial *= i;
    return factorial;
```

n	n!
0	1
1	1
2	2 * 1 = 2
3	3 * 2 * 1 = 3 * 2 = 6
4	4 * 3 * 2 * 1 = 4 * 6 = 24
5	5 * 4 * 3 * 2 * 1 = 5 * 24 = 120
6	6 * 5 * 4 * 3 * 2 * 1 = 6 * 120 = 720
7	7 * 6 * 5 * 4 * 3 * 2 * 1 = 7 * 720 = 5040
...	...

Aplicaciones no numéricas

- Ejemplos de aplicaciones no numéricas:
  - Búsqueda binaria

Ejemplo 1: Búsqueda binaria

- Cuando los datos están ordenados, la búsqueda resulta más rápida
- Si comparamos el elemento a buscar con el que está justo en la mitad de los datos ordenados, podemos decidir si es ése el elemento que buscamos o debemos continuar buscando, pero solo en la mitad:
  - Derecha o
  - Izquierda



- El mismo proceso se puede repetir con la mitad elegida, comparando con el elemento que se encuentra en el centro de dicha mitad
- En cada comparación, la búsqueda se reduce a comprobar si el dato buscado está entre la mitad de los anteriores
- La búsqueda finaliza cuando el elemento se encuentra, o bien cuando la zona pendiente de examinar queda reducida a un solo elemento (o ninguno) después de sucesivas divisiones en mitades
- Esta búsqueda se denomina *búsqueda binaria* o *dicotómica*

Ejemplo 1.1: Búsqueda binaria recursiva

```
public static int busquedaBinaria(Comparable [] a, Comparable x, int inf, int
    sup) throws ElementoNoEncontrado{
// Localizar elemento x buscado en vector abierto a, entre inf y sup,
// y en su caso indicar su posición

    if (inf > sup)
        throw new ElementoNoEncontrado("Fallo")
    int med = (inf + sup) / 2;
    if (a[med].compara(x) < 0)                    // Buscar en a[med+1..sup]
        return busquedaBinaria(a,x,med+1,sup);
    else if (a[med].compara(x) > 0)  // Buscar en a[inf..sup-1]
        return busquedaBinaria(a,x,inf,med-1);
    else
        return med;        // Elemento x encontrado en a[med]
}
```

inf	med-1	med	med+1	sup
	...		...	

Ejemplo 1.2: Búsqueda binaria iterativa

```
int Indice(T_Elemento buscado,const T_Elemento v[],int N) {
// Localizar cierto elemento buscado en vector abierto v, y en su caso indicar su posición
    int izq,dch,mitad,pos;
    izq = 0;
    dch = N-1;
    pos = -1;
    while (pos < 0 && izq <= dch) {
        mitad = (izq + dch) / 2;
        if (v[mitad] == buscado) {
            pos = mitad;
        else if (v[mitad] < buscado) {
            dch = mitad - 1;
        } else { /* v[mitad] > buscado */
            izq = mitad + 1;
        }
    }
    return pos;
}
```

Vector v					
0	1	2	...	N-2	N-1
v_0	v_1	v_2	...	v_{N-2}	v_{N-1}

Aplicaciones numéricas

- Ejemplos de aplicaciones numéricas:
  - Exponenciación
  - Aritmética modular
    - Exponenciación modular
  - Máximo común divisor (m.c.d.)
    - Inversos multiplicativos

Ejemplo 1: Cálculo de la función exponencial (exponenciación)

- Definiciones recursivas de la **potencia n-ésima de x, x^n**:
  - **Definición 1:**
    - 1, si n = 0
    - $x.x^{n-1}$, si n > 1
  - **Definición 2:**
    - $(x^{n/2})^2$, si n es par
    - $x.x^{n-1}$, si n es impar
  - **Definición 3:**
    - $(x^{n/2})^2$, si n es par
    - $x.(x^{(n-1)/2})^2$, si n es impar

Estrategias de programación

Ejemplo 1.1: Función potencia recursiva

```
public static long potencia(long x, long n) {
// Calcula la potencia n-ésima de x, x^n

   if (n == 0)
        return 1;
   else
        return x * potencia(x,n-1);
}
```

Ejemplo 1.2: Función Potencia iterativa

```
public static long potencia(long x, long n) {
// Calcula la potencia n-ésima de x, x^n
   long c = 1;
   for (int k = 1; k <= n; k++)
        c *= x;
    return c;
}
```

Aritmética modular

- El *operador módulo %* en Java calcula el resto de la división entre dos valores de tipo entero (**int** o **long**)
- Es decir, n % m $\in$ {0, 1, 2, …, m-1}
- **Ejemplos:**
  - 7 % 2 = 1 (7 = 2 x 3 + 1)
  - 36 % 5 = 1 (36 = 5 x 7 + 1)
  - 37 % 5 = 2 (37 = 5 x 7 + 2)
  - 38 % 5 = 3 (38 = 5 x 7 + 3)
  - 39 % 5 = 4 (39 = 5 x 7 + 4)
  - 40 % 5 = 0 (40 = 5 x 8 + 0)

Ejemplo 2: Cálculo de la función exponenciación modular

- Definición recursiva de la **potencia n-ésima de x módulo p**

$$x^n \bmod p = \begin{cases} (x^2 \bmod p)^{n/2}, \text{ si n es par} \\ x.(x^2 \bmod p)^{n/2} \bmod p, \text{ si n es impar} \end{cases}$$

Ejemplo 2.1: Función potencia modular recursiva

```
public static long potenciaMod(long x, long n , long p) {
// Calcula la potencia n-ésima de x módulo p, x^n mod p

   if (n == 0)
        return 1;
   else
        long tmp = potenciaMod((x * x) % p, n / 2, p);
        if (n % 2 != 0)                          // n impar
                tmp = (x * tmp) % p;
        return tmp;
}
```

Algoritmo de Euclides

- El *algoritmo de Euclides* es usado para calcular el *máximo común divisor (m.c.d.)* entre dos números enteros positivos p y q
- Es decir, es el mayor entero tal que:

  divide a p y a q

- **Ejemplo:**

  mcd(42,36) = 6, ya que 42 = 6 x 7 y 36 = 6 x 6

Ejemplo 3: Máximo común divisor

- **Definición** (recursiva para calcular el máximo común divisor)**:**
  - mcd(p,0) = p
  - mcd(p,q) = mcd(q,p mod q)
- **Ejemplo:** Aplicación de la definición a mcd(42,36)

mcd(42,36) =
mcd(36,42 % 36) =
mcd(36,6) =
mcd(6,36 % 6) =
mcd(6,0) =
6

Ejemplo 3.1: Función mcd recursiva

```
public static long mcd(long p, long q) {
// Calcula el máximo común divisor (m.c.d.)
// entre dos números enteros positivos p y q

   if (q == 0)
        return p;
   else
        return mcd(q,p % q);
}
```